

## CSCI 311.S26 - CityConnections - Team 4

Aum Mehta (ajm057@bucknell.edu)  
Ben Khant (ak058@bucknell.edu)  
Patrick Gardner (pjpg017@bucknell.edu)  
Laura Ozoria (lgom001@bucknell.edu)

April 2026

In this project, we model a city's road network as a graph, where the intersections are represented as nodes and the roads are represented as edges with weights corresponding to their lengths. Our goal is to find the minimum amount of cable needed to connect the entire city, while ensuring that every node is reachable and making sure we are minimizing the total edge weight. This problem can be solved using minimum spanning tree algorithms, which efficiently find a subset of edges that connects all nodes without cycles and with the lowest possible total cost.

The input for this project is given as a file where each line represents an edge, including an edge ID, a start node, an end node, and a weight corresponding to the distance. We use this data to construct a graph representation of the city's road network, where nodes represent intersections and edges represent roads. We read the input file line by line, skipping any lines that begin with a `#` symbol. For each valid line, we parse the edge ID, start node, end node, and weight as integers and a float respectively. We store all edges in a list of tuples of the form `(edge_id, start, end, weight)` for easy access during output. Simultaneously, we build an adjacency list using a dictionary, where each key is a node ID and its value is a list of tuples containing its neighboring node, the edge weight, and the edge ID. Since roads are bidirectional, each edge is added to both the start node's and end node's lists. This adjacency list allows our algorithms to efficiently look up all neighbors of any given node in  $O(\text{degree})$  time. This representation allows us to efficiently run our algorithms on the graph.

Our first implementation uses Kruskal's algorithm to compute the minimum spanning tree. The idea is to sort all edges from smallest to largest weight, and then go through them one by one, adding an edge only if it does not create a cycle. To check this, we use a Union-Find data structure. For each edge, we check if its two endpoints are already connected using the `find` function. If they are in the same set, we skip the edge since it would form a cycle. Otherwise,

we connect the two sets using `union` and add the edge to the tree. We also use path compression and union by rank to make these operations faster. The overall runtime of Kruskal’s algorithm is  $O(E \log E)$ , mainly due to sorting the edges.

Our second implementation uses Prim’s algorithm to compute the minimum spanning tree. This algorithm works by starting from a single node and repeatedly adding the smallest edge that connects the current tree to a new node. Our implementation tracks visited nodes using a set  $S$ , which nodes are added to the MST. We used a `MinPriorityQueue`, initialized to  $\infty$  for all nodes except the starting node, which gets a key of 0. On each step, we call `extract_min` to get the unvisited node with the cheapest connection cost. Once a node  $v$  is extracted, we add its edge to the MST and mark it as visited by adding it to  $S$ . Then, we examine all of  $v$ ’s neighbors with an adjacency list. The runtime of this algorithm is  $O((|V| + |E|) \log |V|)$ .

Both implementations correctly produce a minimum spanning tree that connects all nodes with the smallest possible total edge weight. While they solve the same problem, they take different approaches and use different data structures. As shown in Figure 1, Kruskal’s algorithm significantly outperformed Prim’s across all datasets. On the smallest dataset (Oldenburg, 7,035 edges), Kruskal ran in 28.14ms compared to Prim’s 82.96ms, roughly a 3 times difference. This expanded with larger inputs, on San Francisco’s (223,001 edges), Kruskal completed in 847.25ms while Prim took 5,238.39ms, nearly a 6 times difference. Although both algorithms have similar asymptotic complexities, the performance difference comes down to what each algorithm actually does in practice. Kruskal’s is a single sort of all edges, whereas Prim’s requires repeated heap updates for every neighbor of every visited node, which adds up to significantly more later.

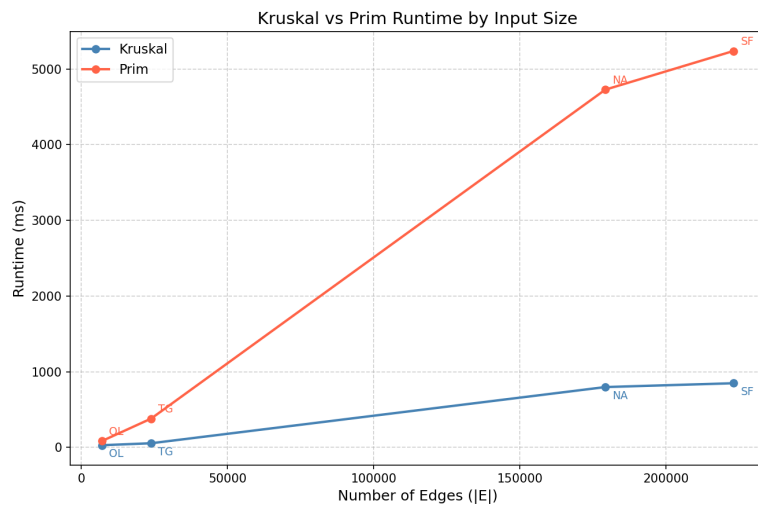


Figure 1: Runtime comparison of Kruskal's and Prim's algorithms across four different datasets.